

RESEARCH SEMINAR

SimToolReal

An Object-Centric Policy for
Zero-Shot Dexterous Tool Manipulation

Can one reusable controller
turn tool motion into robot skill?

Kushal Kedia* · Tyler Ga Wei Lum*
Jeannette Bohg · C. Karen Liu
Cornell University · Stanford University

RSS 2026 · Based on arXiv v2, 24 February 2026

Real



How would you teach a robot to use tools?

Suppose the robot must use
a new brush, hammer or spatula.
What would you build first?

Collect robot demonstrations

Learn a policy from action supervision.

Use a human demonstration

Transfer the motion across embodiments.

Learn through simulation

Let physical interaction shape the controller.

What makes demonstration-based tool use difficult?

Thin-object grasping, in-hand rotation and external forces make contact quality important.

Robot action demonstrations

- High-quality dexterous teleoperation is difficult.
- Limited force feedback makes stable contacts hard to demonstrate.

Human motion retargeting

- Human and robot fingers have different kinematics.
- Matching hand motion does not ensure a physically stable grasp.

What if a demonstration specified tool motion, while the robot learned how to realize it?

Tool use as a sequence of pose-reaching problems

TASK SPECIFICATION

Human tool motion

6D pose waypoints



Reusable goal-reaching
RL policy



ROBOT EXECUTION

Arm + hand actions

Specify where the tool goes

3D position + 3D orientation

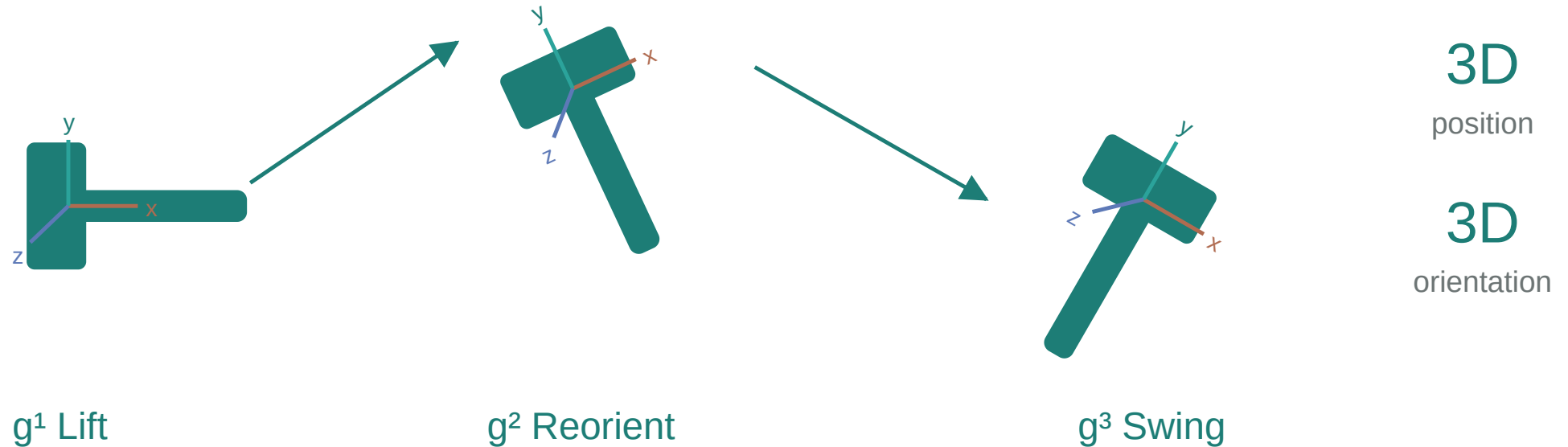
Learn how this robot moves it

Contacts and regrasping emerge from RL

Object goals specify the motion. RL discovers robot-specific grasps and in-hand actions.

A 6D pose specifies position and orientation

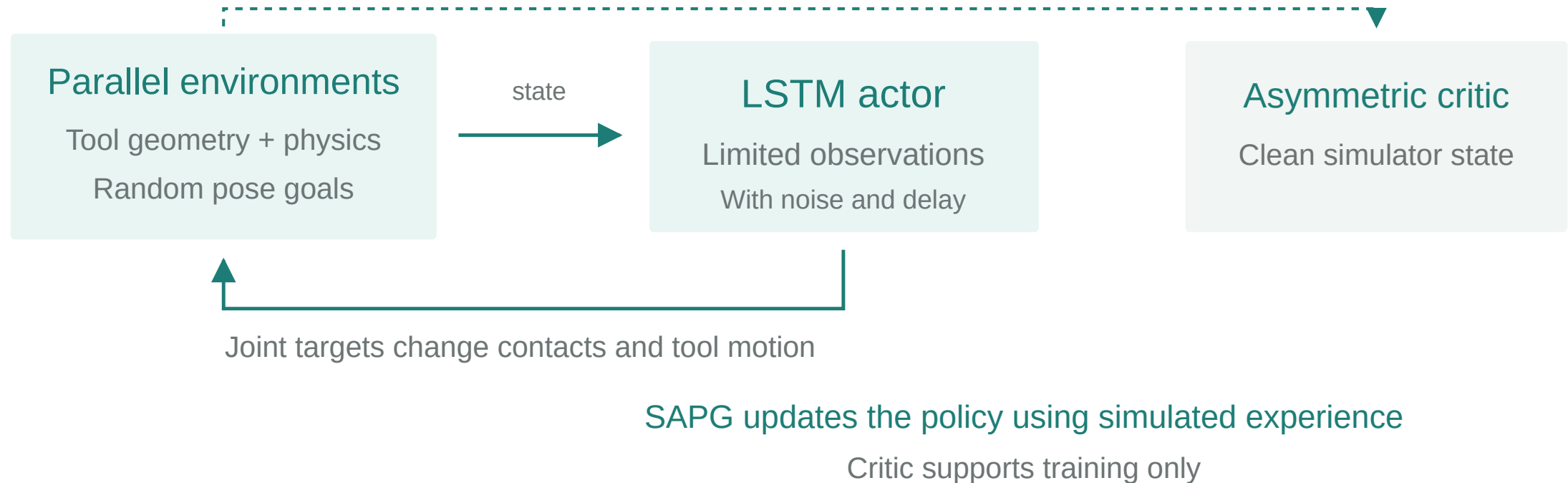
A task becomes an ordered set of rigid-object configurations



$$g^k = (R^k, p^k) \in SE(3), \quad \mathcal{G} = \{g^1, \dots, g^K\}$$

Training pipeline: learn pose reaching through interaction

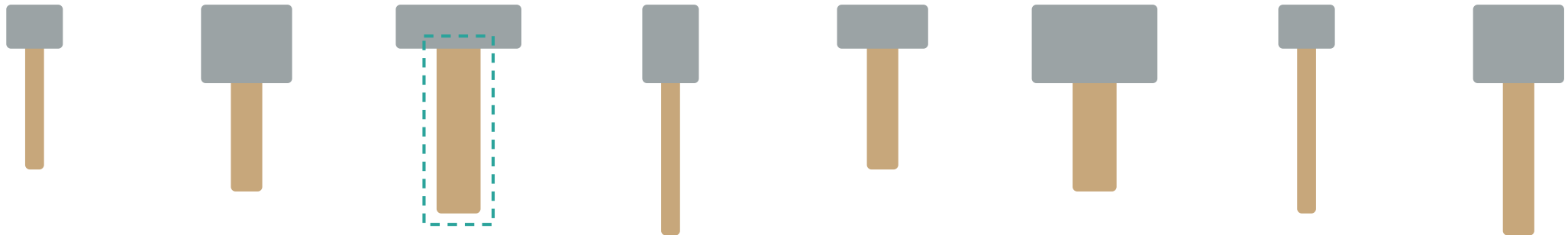
SIMULATION ONLY



Procedural tools and random goals supply the training distribution. No human task trajectories enter this loop.

Procedural tools cover shape and mass variation

One structural prior, many physical instances



Geometry

Shape + size

Mass distribution

Handle / head densities

Shared policy inputs

Pose + grasp-region box

The commonality is a geometric interface: tool pose and a graspable region.

A shared reward induces the component skills

$$r = r_{\text{smooth}} + r_{\text{grasp}} + \mathbb{I}_{\text{grasped}} r_{\text{goal}}$$

01 Approach and lift

Shaping helps the hand acquire the object from the table.

02 Reach a pose

Reward improvement in pose distance and give a success bonus.

03 Repeat

After success, sample another goal with the same reward.

“Universal objective” still includes engineered grasping and smoothness terms.

The policy observes a compact, object-centric state

Robot proprioception

- Joint positions and velocities
- Previous joint-position targets
- Palm pose and palm-relative fingertip positions

Tool and current goal

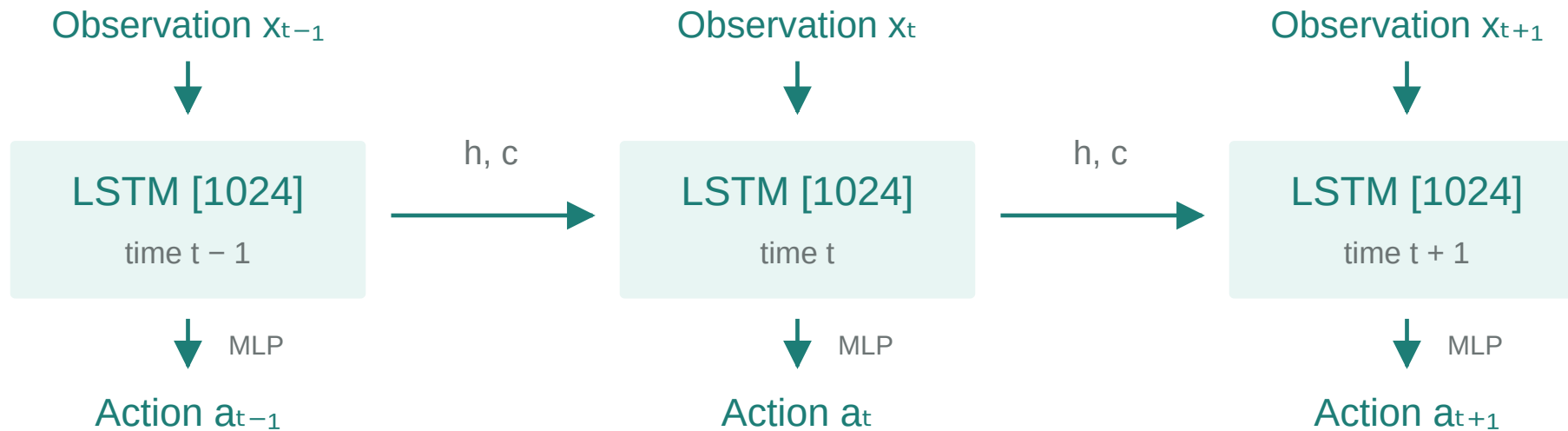
- Tool orientation and palm-relative keypoints
- Four keypoint errors to the goal
- Dimensions of the grasp-region box

$$x_t = \text{features}(s_t, o_t, \phi, g^k)$$

Relative keypoints encode tool geometry and the pose error in a reusable coordinate system.

LSTM memory carries interaction history across steps

THE SAME RECURRENT NETWORK AT EVERY CONTROL STEP



MLP action head: $1024 \rightarrow 1024 \rightarrow 512 \rightarrow 512 \rightarrow 29$ outputs

The hidden state h and cell state c preserve history when geometry and dynamics are only partly observed.

The action head controls the arm and hand differently

$$a_t = [a_t^{\text{arm}}, a_t^{\text{hand}}] \in [-1, 1]^{29}$$

7 arm joints: incremental targets

$$\hat{q}_t^{\text{arm}} = q_{t-1}^{\text{target, arm}} + 0.025a_t^{\text{arm}}$$

Each action adjusts the previous target.

22 hand joints: absolute targets

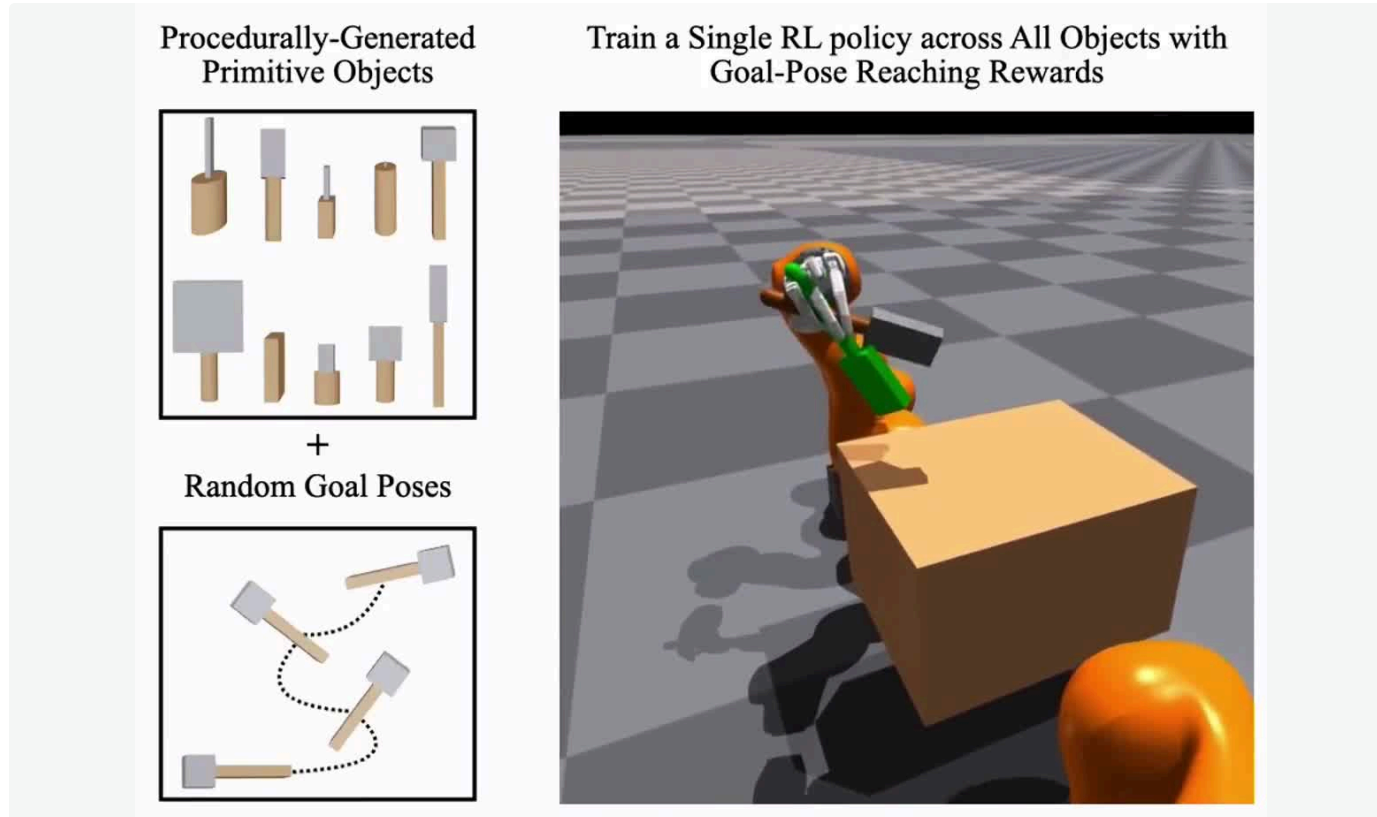
$$\hat{q}_t^{\text{hand}} = q_{\min} + \frac{a_t^{\text{hand}} + 1}{2} \odot (q_{\max} - q_{\min})$$

Each action maps into the joint's allowed range.

Joint limits and EMA smoothing ($\alpha = 0.1$) produce the final targets at 60 Hz.

The deployed actor keeps its weights fixed. Recurrent state updates at every control step.

Simulation: reaching diverse tool poses



WHAT TO WATCH

The **green mesh** indicates the current target pose.

The policy learns to acquire and reorient procedurally generated tools.

There is no human task trajectory in this training loop.

SimToolReal project: training environment · 1× speed · silent excerpt

[Original project video ↗](#)

Simulation exposes the state. Reality requires estimation.

Inside the simulator

- Exact tool pose and geometry are available.
- Random goal poses define the task.
- Noise and delays train the actor on imperfect inputs.

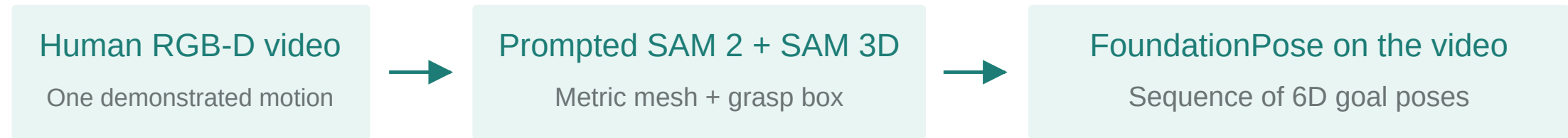
On the real robot

- The tool pose is hidden behind pixels and occlusion.
- A human demonstration supplies a meaningful trajectory.
- Perception must recover the same policy interface.

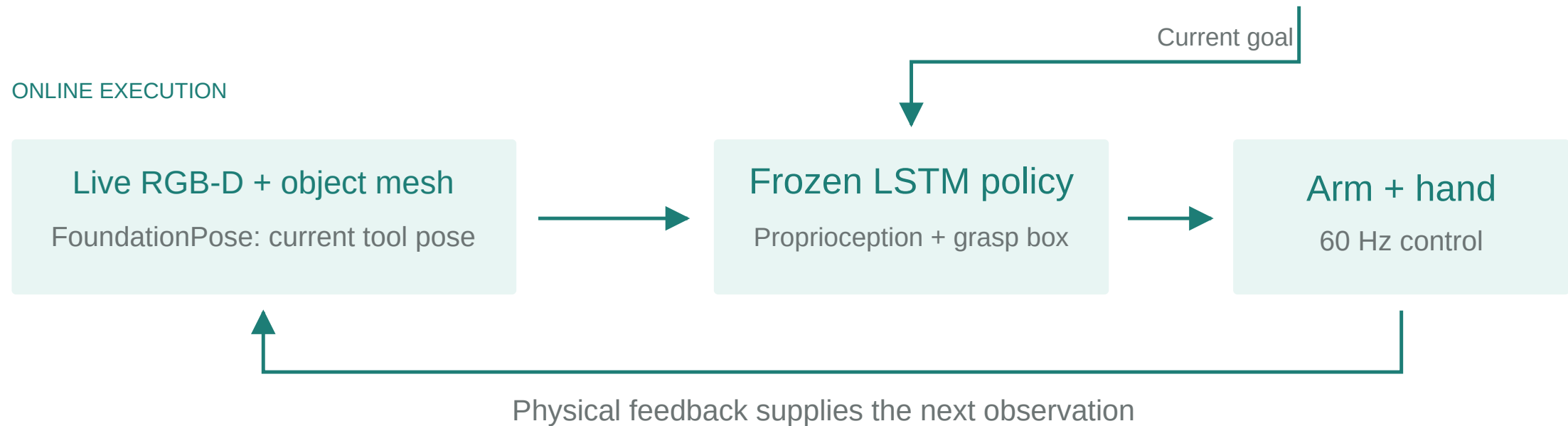
Keep the motor policy fixed. Replace simulator-derived inputs with measured and estimated inputs.

Real deployment supplies the same policy interface

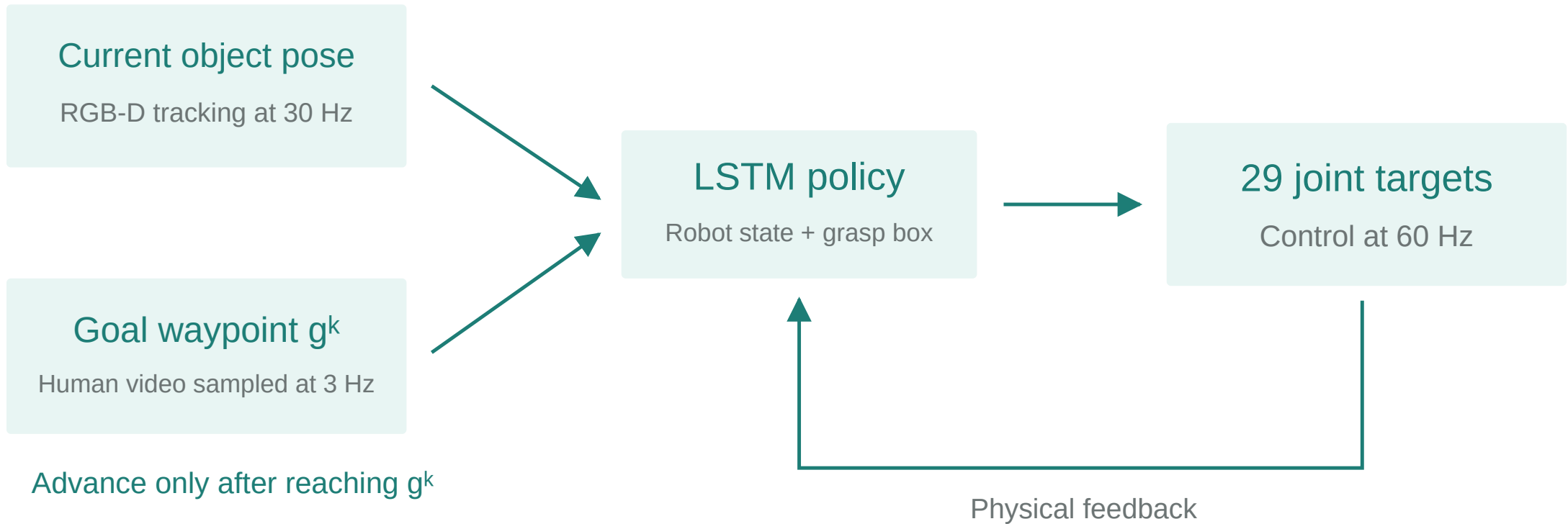
SETUP FOR A NEW TOOL / DEMONSTRATION



ONLINE EXECUTION

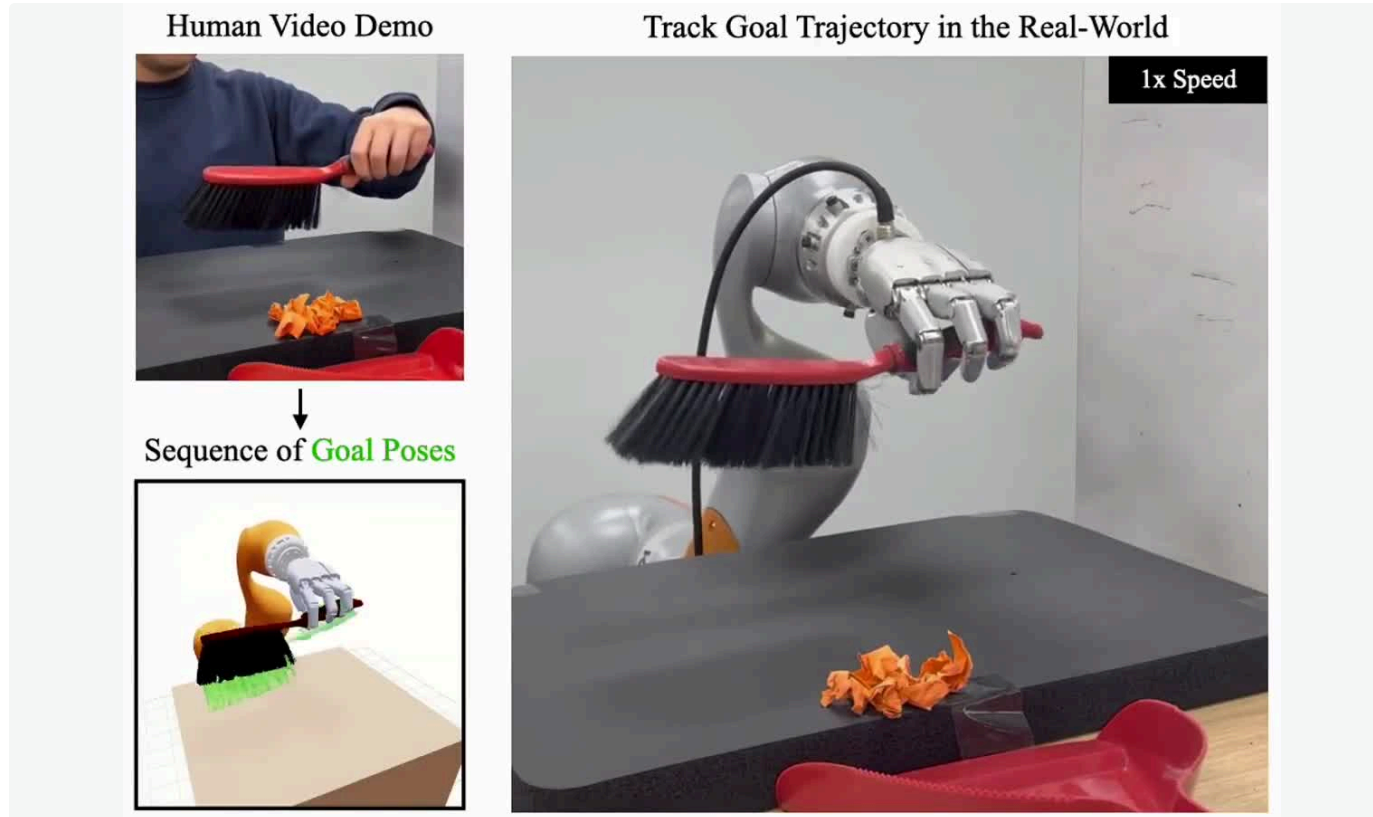


The robot advances goals only after reaching them



3 Hz is waypoint sampling. Goal switching is event-driven, while the controller runs at 60 Hz.

The closed loop follows unseen tool trajectories



SimToolReal project: real-world inference · 1x speed · silent excerpt

[Original project video ↗](#)

WHAT TO WATCH

The human demonstration supplies the **tool-motion goals**.

The fixed policy grasps, reorients and moves the real brush.

The goal interface specifies tool motion rather than robot finger trajectories.

Brush and hammer: reorientation before interaction

Brush

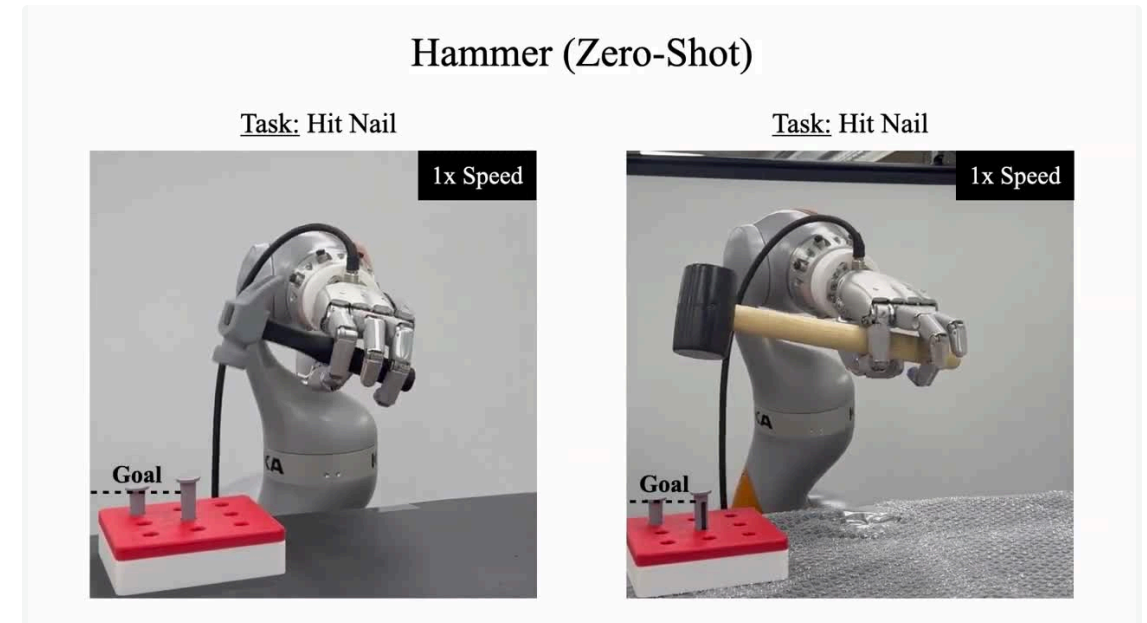


Brush · 1× speed · silent excerpt

[Original project video ↗](#)

Reorient the handle, then sweep along the surface.

Hammer



Hammer · 1× speed · silent excerpt

[Original project video ↗](#)

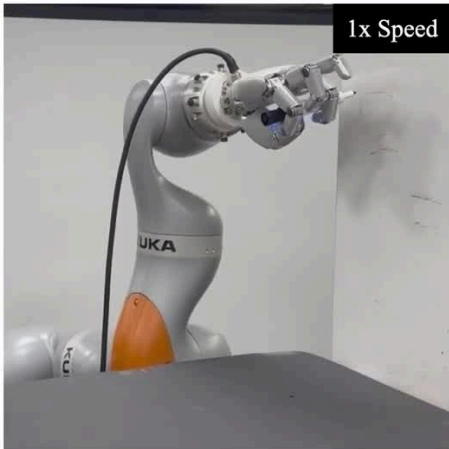
Change the grasp-tool orientation before the swing.

Marker and eraser: tool motion along a surface

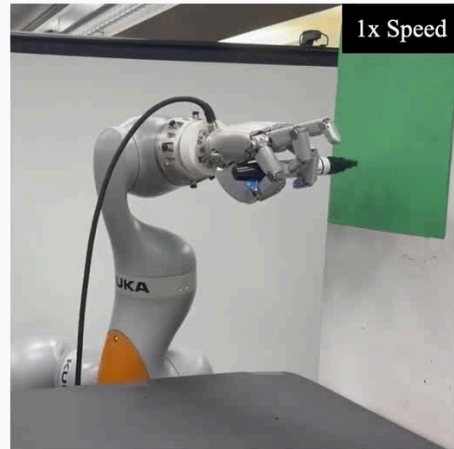
Marker

Marker (Zero-Shot)

Task: Draw Smiley



Task: Draw Smiley



Marker · 1× speed · silent excerpt

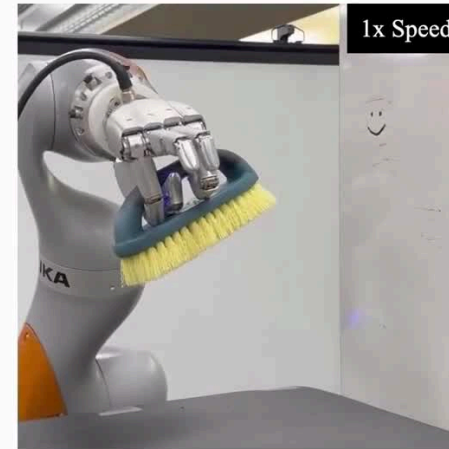
[Original project video ↗](#)

Acquire a thin tool and move its tip across the board.

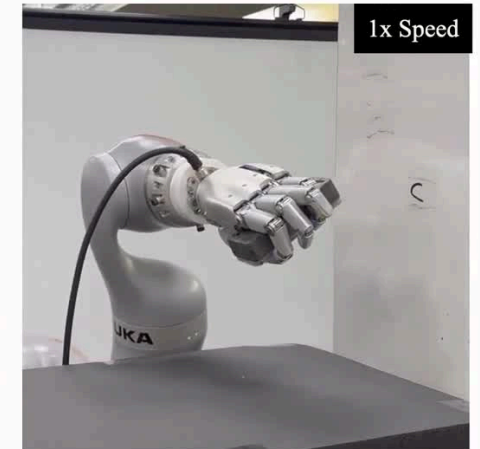
Eraser

Eraser (Zero-Shot)

Task: Wipe Whiteboard



Task: Wipe Whiteboard



Eraser · 1× speed · silent excerpt

[Original project video ↗](#)

Maintain a grasp while following a wiping path.

Spatula and screwdriver: larger orientation changes

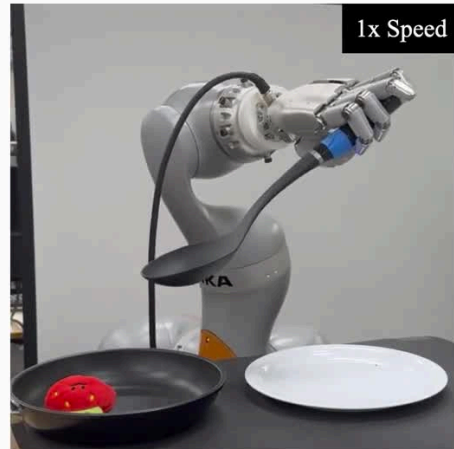
Spatula

Spatula (Zero-Shot)

Task: Flip Toast



Task: Serve Strawberry



Spatula · 1× speed · silent excerpt

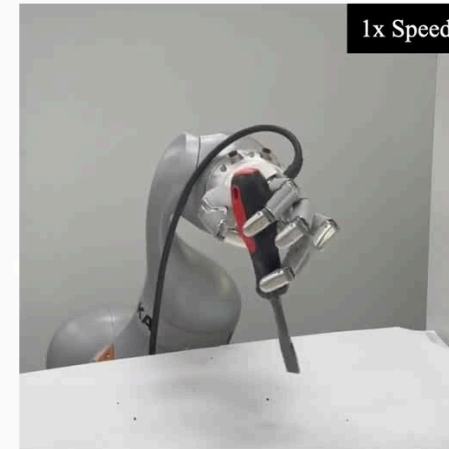
[Original project video ↗](#)

Serving and flipping demand changing tool orientation.

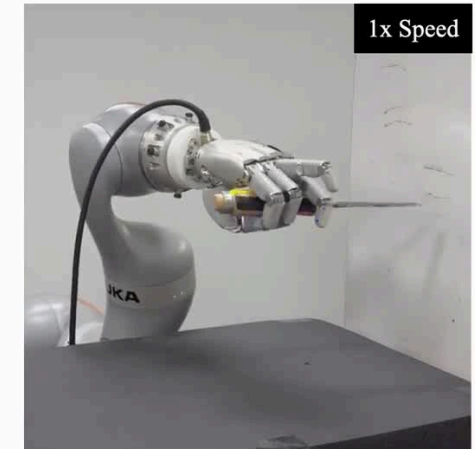
Screwdriver

Screwdriver (Zero-Shot)

Task: Spin Vertically



Task: Spin Horizontally



Screwdriver · 1× speed · silent excerpt

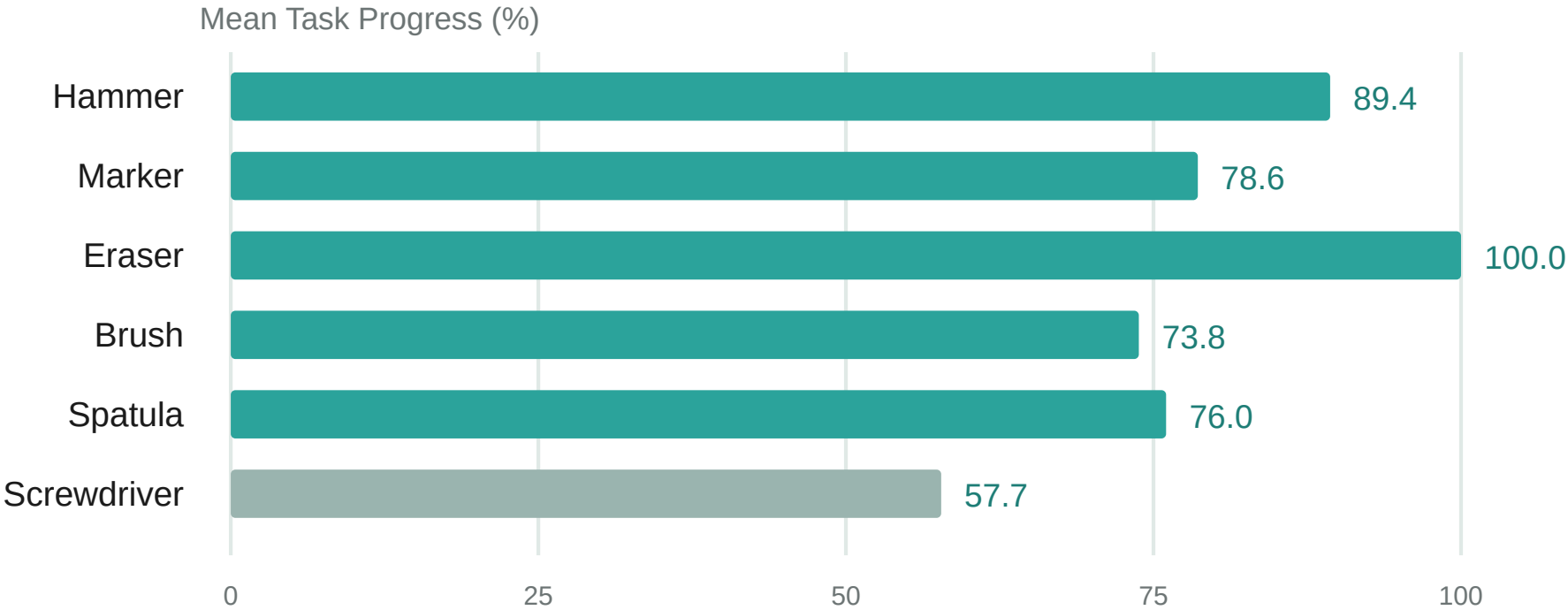
[Original project video ↗](#)

Reorient and spin the tool in free space.

Real-world transfer across six tool categories

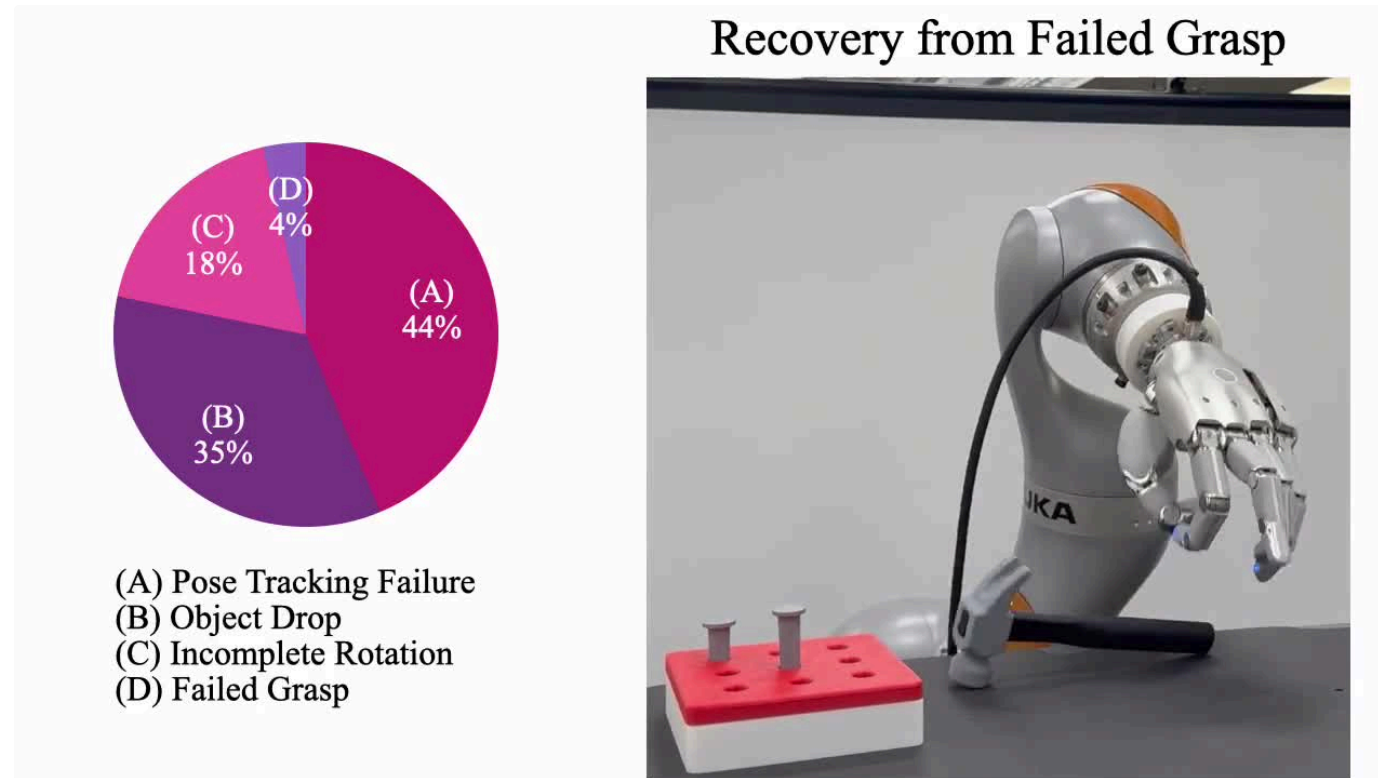
12 objects · 24 object–task variations · 120 rollouts

Task Progress = fraction of sequential pose goals reached.



Mean progress: **79.3%**. This metric does not establish functional task success.

Recovery illustrates feedback, with clear limits



SimToolReal project: recovery behavior · 1× speed · silent excerpt

[Original project video ↗](#)

WHAT TO WATCH

The policy can attempt to **regrasp** after an error.

Recovery depends on the tool staying **reachable and trackable**.

Rigid pose goals leave force objectives, clutter and task replanning unresolved.

Three research directions suggested by the method

PRESENTER HYPOTHESES · NOT RESULTS ESTABLISHED BY THIS PAPER

01 Reusable skills beyond end-to-end action scaling

Structured RL could supply motor skills for a higher-level system.
Online skill learning in the wild remains an open step.

02 Compact interfaces for other interactions

Other tasks may admit a useful abstraction, but pose alone may be insufficient.
Contact, force, articulation or shape may need to enter the state.

03 Lightweight VLMs for semantic setup

A VLM could select tools, grasp regions and subgoals.
Metric, low-latency pose tracking still needs geometric grounding.

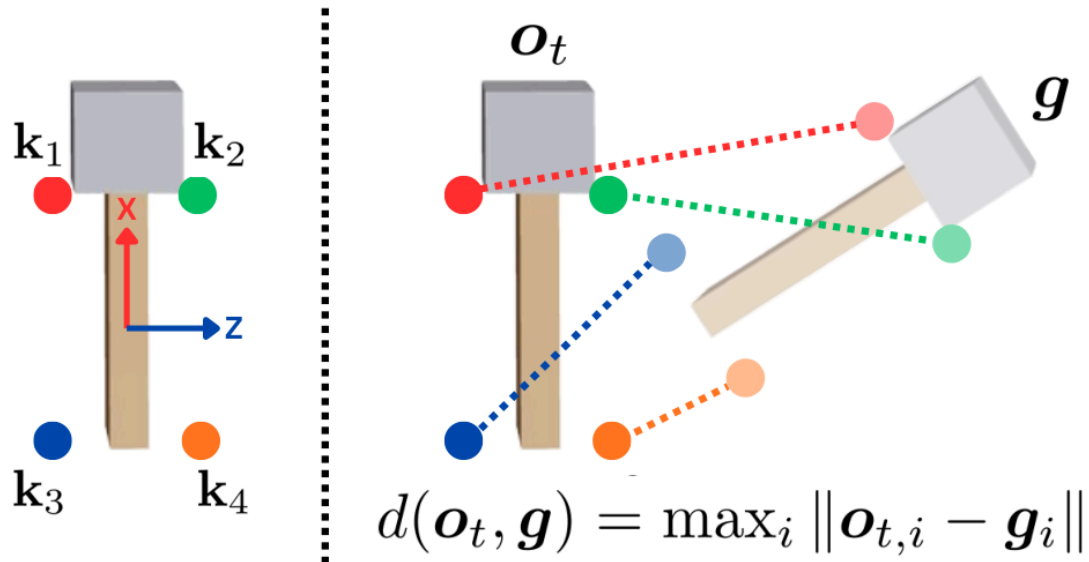
A reusable motor skill needs a well-chosen task interface

- 01 Specify motion at the object level**
Human video supplies goals without prescribing robot finger motion.
- 02 Learn the controller over varied physics**
Procedural primitives and random goals support policy reuse.
- 03 Preserve the boundary of the evidence**
Strong trajectory transfer still leaves perception, force and replanning open.

Which parts of a manipulation task can we safely compress into a reusable goal interface?

BACKUP

Pose error couples translation and rotation



Paper Fig. 9 · Four object-frame pose keypoints

$$d(o_t, g) = \max_{i=1,\dots,4} \|o_{t,i} - g_i\|$$

- Reward keypoints use fixed scales across objects.
- Observation keypoints depend on the object instance.
- Training tolerance: 1 cm. Evaluation: 2 cm.

A “2 cm pose error” is a keypoint-space criterion, not a separate orientation threshold.

BACKUP

In-hand control matters in the brush comparison

No initial tool rotation



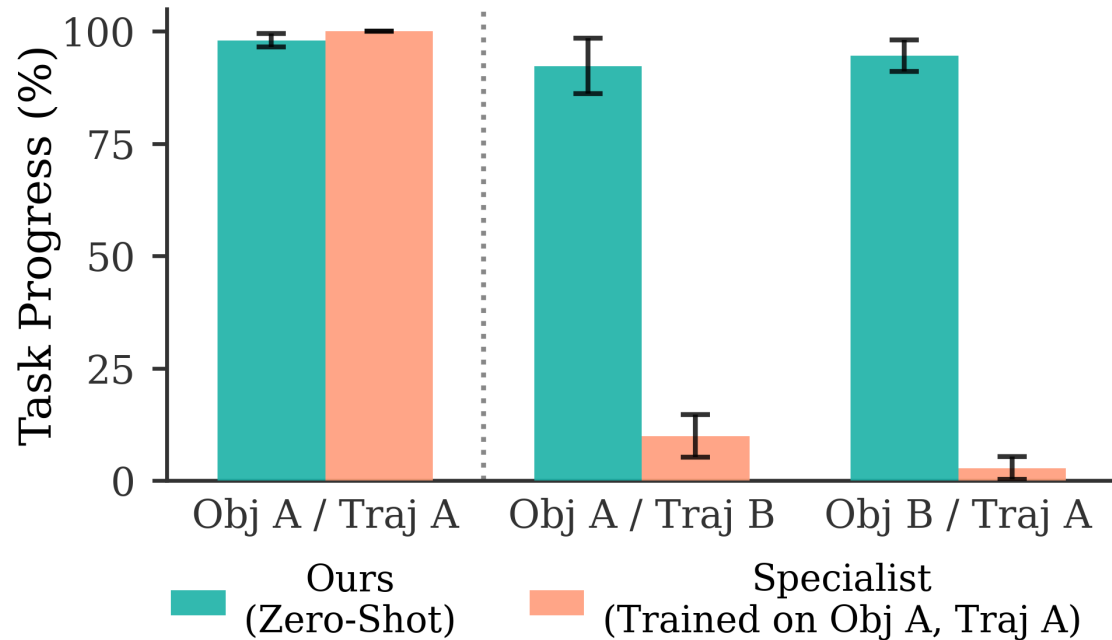
90° tool rotation required



Five rollouts per variation. Same red-brush task, two initial orientations. Values from Fig. 5.

BACKUP

Specialists are sensitive to changed tools and goals

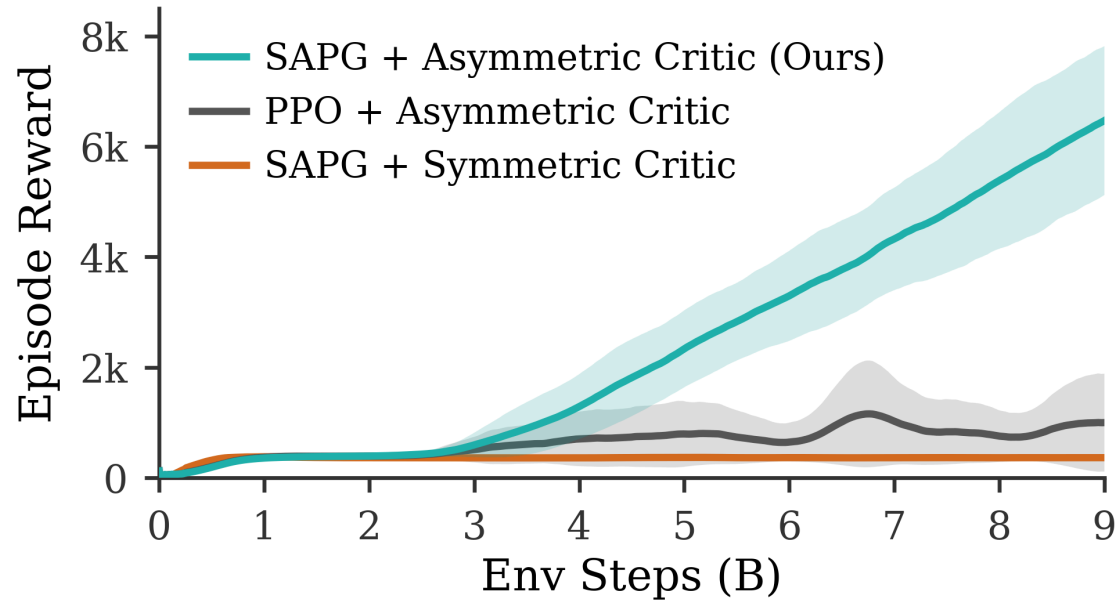


Paper Fig. 6 · Comparison to specialist RL policies

- Match specialist performance on its training setup.
- Keep high progress when either the tool or the trajectory changes.
- All three conditions are evaluated in simulation.

BACKUP

Ablations identify important RL ingredients



Paper Fig. 8 · Training reward, mean over five seeds

- Replacing SAPG with PPO lowers training reward.
- Removing the asymmetric critic severely hinders learning.
- These curves cover 9B environment steps.

This ablation measures training reward, not real-world functional task completion.

References, videos and source material

Main paper

Kedia, Lum, Bohg and Liu. *SimToolReal: An Object-Centric Policy for Zero-Shot Dexterous Tool Manipulation*.
arxiv.org/abs/2602.16863v2 · [RSS 2026 proceedings](#)

Project and code

simtoolreal.github.io · github.com/tylerlum/simtoolreal

Closely related work

Chen et al. *Object-Centric Dexterous Manipulation from Human Motion Data*. CoRL 2024.

Lum et al. *Crossing the Human-Robot Embodiment Gap with Sim-to-Real RL Using One Human Demonstration*. 2025.

Petrenko et al. *DexPBT*. 2023.

Design reference

[Mizoreww/steerable-vla-talk](https://mizoreww.github.io/steerable-vla-talk) and the previous A2A seminar deck.